Check for updates

Received: 12 April 2021 | Revised: 5 November 2021 | Accepted: 22 November 2021

DOI: 10.1111/exsy.12924

ORIGINAL ARTICLE

Expert Systems WILEY

# Modelling and analysing the reliability for microservice-based cloud application based on predicate Petri net

Zheng Liu[1] | Guisheng Fan[1] | Huiqun Yu[1] | Liqiong Chen[2]

[1]Department of Computer Science and Engineering, East China University of Science and Technology, Shanghai, China

[2]School of Computer Science and Information Engineering, Shanghai Institute of Technology, Shanghai, China

**Correspondence**
Guisheng Fan and Huiqun Yu, Department of Computer Science and Engineering, East China University of Science and Technology, Shanghai, China.
Email: gsfan@ecust.edu.cn and yhq@ecust.edu.cn

**Funding information**
NSF of China, Grant/Award Number: 61772200

## Abstract

Microservice design is a new paradigm of cloud application development. Different from monolithic design, microservice enjoys merits of fine-grained and loosely coupled services, and it is becoming more and more popular. The application developed with microservice has a good advantage in independent development and flexible deployment, especially for complex distributed systems. However, there is a big gap between the reliability requirements and microservice-based cloud applications. This article proposes a reliability model of microservice-based cloud application by using predicate Petri net. First, a microservice reliability requirement is given, some basic concepts of predicate Petri net are defined with syntax and semantics. Second, a microservice reliability strategy is proposed, which uses microservice instances and circuit breaker to improve the reliability of the system. Based on the constructed microservice reliability model, the correctness of predicate Petri net modelling and the effectiveness of the strategies are proven theoretically. Finally, an example is given to illustrate the establishment and analysis process of the model, and several groups of experiments are carried out to verify the effectiveness and feasibility of the method. Experimental results show that the proposed microservice reliability strategy is effective.

**KEYWORDS**
cloud computing, microservice architecture, predicate Petri net, software reliability

## 1 | INTRODUCTION

Cloud applications are internet-based programs in which some or all of the processing logic and data are stored and processed in the cloud. Users interact with applications through web browsers or mobile applications, and data processing is managed by a combination of local devices and cloud computing solutions (Gao et al., 2020). From the perspective of users, the behaviour of cloud applications is similar to that of standard websites, and their computing and data processing are processed in the cloud. However, accessing the cloud resources of the application means that users can obtain more computing power and bandwidth than the actual situation on site (Wu et al., 2018). In terms of background processing, microservice is an architectural method to create cloud applications. Each application is built as a set of services, each running in its own process and communicating through the application programming interface. These services are small applications that can be deployed independently, easy to integrate, and can be deployed on multiple servers (Lyu et al., 2020).

Microservice architecture (MSA) is a design style that divides large and complex applications into microservices with the core idea of 'divide and conquer' rule. As a solution to complex problems, it effectively improves resource utilization, realize the separation of concerns, and make each part of decomposition highly cohesive and loosely coupled (Li & Guo, 2015). With the research on Quality of Service (QoS), MSA shows great potential through the efficient use of cloud resources. The cloud application with MSA breaks the limitations of computing resources and

 © 2021 John Wiley & Sons Ltd. 

exchange microservice between platforms (Petrasch, 2017). However, microservice-based cloud application faces challenges of reliability requirement, such as minimizing the impact of failure, isolating failure cascading and post failure availability (Aderaldo et al., 2017). To solve this challenge, this article studies how to reduce the impact of microservice failures and improve the reliability of cloud applications. The microservices reliability and cascading effects of failure deserve to be studied.

The predicate Petri net (PrT net) is a modelling language for concurrent control flow, which has been widely used to describe discrete event distributed systems. The analysis of PrT net is helpful to obtain the structure properties and dynamic behaviour of the system (Wu et al., 2020). In the MSA, each service can be deployed in a separate execution environment to handle multiple concerns (Zhou et al., 2019). Since cloud applications can be scaled up or down more easily independent of physical resources, this provides the possibility of higher reliability and more flexible microservice composition (Gu et al., 2021). This article overcomes the shortcomings of existing modelling methods. By using the principle of PrT net concurrency control and the core idea of divide and conquer of MSA, this article models the microservice based cloud application, and analyses its reliability. First order predicate logic can be used to describe the properties of PrT net model and verify the model.

The main contributions are as follows:

1. PrT net is used to model microservice based cloud applications, including microservices, dependencies between microservices, microservice instances and so on. The state reachability and effectiveness are verified based on the properties of PrT net.
2. A reliable composition strategy of microservice based cloud applications is proposed. The use of microservice instance redundancy mode and circuit breaker integration limits the impact of microservice failure and improves the reliability of cloud applications.
3. Design simulation experiments to analyse the model and strategy from the aspects of microservice redundancies and fault tolerance. The experimental results show that the proposed strategy can achieve the desired target.

The rest is as follows: Section 2 introduces the related work. Section 3 designs the reliability requirement of the microservice-based cloud application. The proposed method is illustrated in detail in Section 4. Section 5 demonstrates the verification process and strategy. The simulated results and discussion are in Section 6. Finally, Section 7 summaries the article.

## 2 | RELATED WORK

Microservice is a hot spot in the research of software services. Many works have proposed some related methods. The advantages and disadvantages of the architectures are compared with the same results obtained by monolithic and microservice architectures (Villamizar et al., 2016). Mayer and Weinreich provide an approach to extract and analyse a microservices-based software architecture based on combined static service information with runtime information (Chaudhry et al., 2020). In Hussain et al. (2020) an adaptive description language Web Services Description Language is proposed to describe adaptive logic at multiple levels, and a supporting microservice platform is given by extending Kubernetes. Noor et al. (2019) propose a general monitoring framework to monitor the performance of microservices deployed across heterogeneous virtualization platforms in a multi-cloud environment. They use bookstore applications implemented on AWS and azure to verify the effectiveness and efficiency of the approach. The problem solved in Dai et al. (2020) integrates components in Cyber-Physical-Social System with choreography-driven microservice composition. Anne et al. (2013) provide analysis algorithms for availability cost optimization and three-dimensional availability performance cost optimization, which provides better results for evolutionary optimization.

The research on the reliability of microservice frameworks has received great attention. Aiming at the reliability evaluation of microservice architecture, Pietrantuono et al. (2018) propose an in vivo testing method to evaluate microservice applications' reliability in operation. Hasselbring and Steinacker (2017) design microservice architectures for scalability and reliability, which can significantly improve the load performance and scalability through fine-grained decomposition and decentralized data management. A method of automatic fault diagnosis based on statistics and microservice application is studied in (Wang et al., 2020). It estimates the workflow's anomaly and then uses the tree edit distance to compare the current tracking and learning baseline to locate the microservice. Wang et al. (2019) analyse the software reliability as an important index in detail and put forward a reliability testing method as a part of acceptance testing. Komarasamy and Muthuswamy (2015) proposes a new task scheduling algorithm for cloud computing, called adaptive deadline dependent job scheduling algorithm to improve the utilization of the processor without being affected by the cloud environment.

The formal method to describe, develop and verify the microservice software model have been studied by many scholars. G. Fan et al. (2016) propose a systematic method to solve the reliability, running time and fault handling of resource scheduling in cloud computing abstracted the resource scheduling process as a meta-object, and built the PrT net model. In G. Fan et al. (2013), a hierarchical service scheduling network is proposed and the competition process of available services is described by queuing theory. The scheduling model of service composition is formed. Camilli (2020) utilizes the formal method for continuous validation of microservice based applications. It adopts the model driven paradigm and uses PrT net to specify and verify the behaviour of time-dependent workflow. B. Mario et al. (2020) present a formal model specifically tailored

14680394, 2022, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.12924 by Ned Univ of Engineering & Tech, Wiley Online Library on [08/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

for reasoning on the deployment of microservice architectures, which can be used to actually synthesize optimal deployment plans. The mathematical analysis is conducted to improve the reliability and robustness of the design.

Since the microservices use a rich body of small services, one of the most significant current discussions on MSA is reliability vulnerabilities resulted from the increasing complexity of applications. To date, few existing methods are perfect, and many are far from mature. Thus, how to optimize the performance of the fault tolerance for the microservice-based cloud applications is worthy of investigation.

## 3 | RELIABILITY REQUIREMENT AND SYSTEM FRAMEWORK

This section specifies the reliability design problem and analyses the modelling technology in the microservice reliability requirement design.

### 3.1 | Problem statement

The dependencies between microservices are directional, which poses new challenges to the reliability guarantee of cloud applications. For example, the microservice failure may spread to other microservices, or even cause the whole application to crash (Hasselbring & Steinacker, 2017). When the microservice crashes, its failure impact should be limited and the performance of overall application remains stable. In addition, because the granularity of splitting ensures the independence and integrity of microservices, the microservices are developed, deployed and updated independently of each other, it brings reliability design challenges.

An example of dependency between microservices is shown in Figure 1:

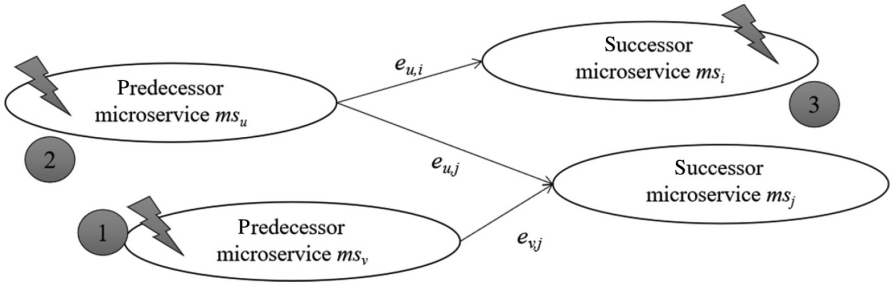



**FIGURE 1** Example of microservice dependency

The $i$-th microservice $ms_i$ is unique and required by others. If there is an edge $e_{i,j}$ between $ms_i$ and $ms_j$, $ms_i$ can be executed after $ms_j$ is completed. Microservice $ms_i$ is called the predecessor microservice of $ms_j$, and $ms_j$ is the successor microservice of $ms_i$. A microservice without a predecessor or a successor is called the entry microservice $ms_{entry}$ or the exit microservice $ms_{exit}$, respectively. The set of predecessor microservices of $ms_i$ are denoted as $PRED(ms_i)$. The $j$-th service instance of the $i$-th microservice is $msi_{i,j}$, and the set of instances is $DE(ms_i)$, $msi_{i,j} \in DE(ms_i)$.

As shown in Figure 1, the cloud application is composed of the microservices $ms$. $\forall ms_u$ and $ms_v$, $ms_u \in (PRED(ms_i) \cap PRED(ms_j))$ and $ms_v \in PRED(ms_j)$. The problems solved in this article can be illustrated as follows:

1. Given that a microservice $ms_v$ is deployed into an instance in set $DE(ms_v)$, how to improve the reliability of $ms_v$?
2. If the predecessor microservice $ms_u$ has failed, how to improve the availability of its successor $ms_j$?
3. How to prevent the fault propagation, since the microservice $ms_i$ fails?

From the analysis mentioned above, the purpose of problem statement 1 is to identify the microservice that needs to be redundant and improve its reliability. Problem statements 2 and 3 are to eliminate the impacts from fault propagation.

### 3.2 | Microservice reliability design and analysis

With microservice-based cloud applications can be decomposed into services with smaller granularity. The idea of divide and conquer rule in MSA is extended to three dimensions, as shown in Figure 2.

14680394, 2022, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.12924 by Ned Univ of Engineering & Tech, Wiley Online Library on [08/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

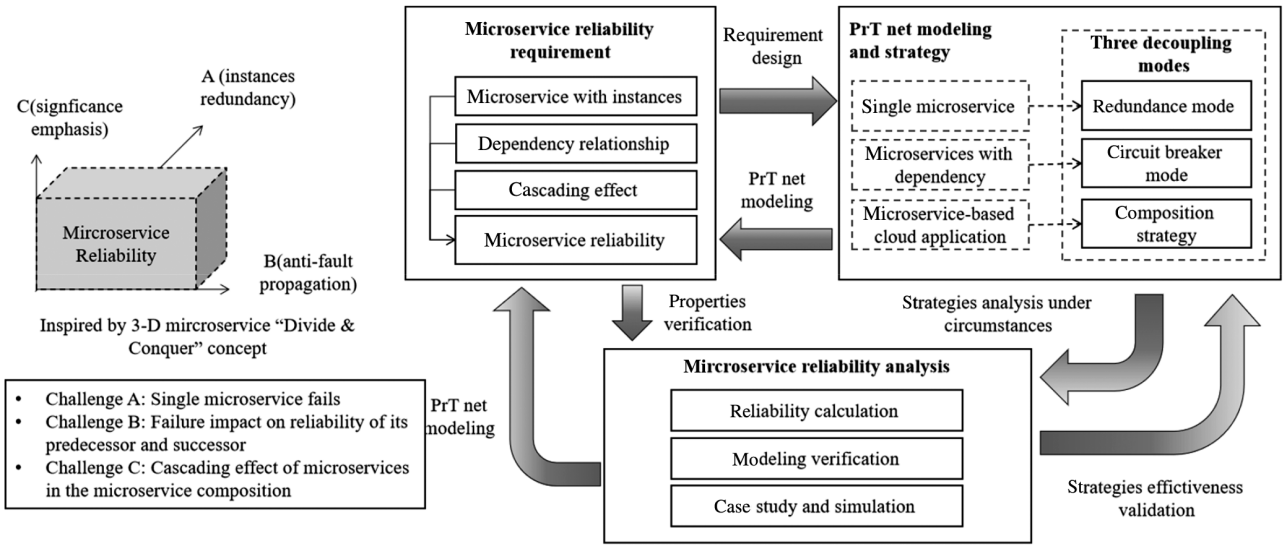



**FIGURE 2** A design and framework of microservice-based application architecture

Clone services improve the reliability of each microservice, just like a stand-in. Considering the overall effectiveness, the service instances should be redundant with in key microservices (Niko et al., 2019). Aiming at the reliability challenges faced by existing microservice cloud applications, this article mainly analyses several aspects, as Figure 2 illustrates. Hence, firstly the requirement of microservice reliability is proposed, which includes microservices, dependency relationship and microservice instances. Secondly, this article improves the reliability of microservices and their composition through PrT net modelling and the proposal of three modes. They are respectively for a single microservice, the dependency of microservices and the composition of microservices. The third part is the model verification and the effectiveness proof of the strategy. The proposed modelling method and strategy are demonstrated through model verification, case analysis and simulation (Fokaefs et al., 2016).

## 3.3 | Microservice reliability requirement

The reliability and availability serve as key decision factors for service design. Reliability refers to the probability that a microservice or microservice composition in a cloud application reaches a certain performance standard and produces correct output within the expected time. In the process of reliability design, the cascading effect of a failure and its impact on microservice-based cloud application should be avoided (Hamzehloui et al., 2018). In this article, availability refers to the percentage of microservices in microservice based cloud application remains running under failure conditions. The reliability improvement strategy of microservices and the availability guarantee of cloud applications based on microservices in case of failure has been study.

A microservice architectural cloud computing contains a microservices set and instance set, among which there is the primary dependency relationship. Multiple instances deploy for each microservice. The reliability of instances may be distinguished because the execution environment varies. This section presents the requirement of microservice reliability, such as microservice set, instance set, reliability and so on.

**Definition 1.** The microservice reliability requirement is a 9 tuple $\Xi = (MS, MSI, C, S, H, RL, DE, CB, REQ)$:

1. $MS$ is the microservice set. Each microservice in $MS$ owns the attribute deadline $d$, which means the microservice should be finished until the required deadline $d$.
2. $MSI$ is the microservice instance set. Each microservice instance $msi_{i,j}$ has a failure rate $\lambda_{i,j}$ and calculation cost $c_{i,j}$.
3. $C$ is the container, which has the million instructions per second (MIPS) as an attribute. The microservice is allocated to the container for execution.
4. $S$ is the state set of the microservice, $S = \{normal, abnormal\}$. $\forall ms \in MS$, state of microservice $ms.s \in S$.
5. $H$ is the state set of microservice instance, $H = \{available, unavailable\}$. $\forall msi \in MSI$, state of microservice $msi.h \in H$.
6. $RL: MS \rightarrow MS^*$ is the dependency relationship among microservices, which includes forward and backward relationships. The predecessor microservice set are denoted as $PRED\ (ms_i) = \{ms_1, ms_2, \ldots, ms_p\}$.

14680394, 2022, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.12924 by Ned Univ of Engineering & Tech, Wiley Online Library on [08/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

7. *DE*: $MS \rightarrow MSI^*$ is the deployment function of microservice, and $DE(ms_i) = \{msi_{i,1}, msi_{i,2}, \ldots, msi_{i,m}\}$ denotes the instance set of microservice $ms_i$, $j$ represents the $j$-th instance of $ms_i$.
8. *CB*: $MS \rightarrow MS$ is the circuit breaker between two microservices. CB has four attributes, namely set of possible current states $State = \{open, semi\text{-}open, closed\}$, maximal failure times $N$, block time $BT$ and sensitivity $K$.
9. *REQ* is the request frequency function of microservices. $\forall\, ms_i$ and $ms_j \in MS$, $REQ(ms_i, ms_j) = freq_{i,j}$.

## 4 | MICROSERVICE RELIABILITY MODELLING BASED ON PREDICATED PETRI NET

This section first gives the basic concepts such as primary service net and the hierarchical microservice composition net based on PrT net, and then models the basic business process, invocation, instance redundancy and circuit breaker of microservice based cloud application.

### 4.1 | PrT net syntax and semantics

As a graphical modelling tool with a rich mathematical foundation, PrT net has been widely used to describe concurrent and distributed systems with loosely coupled characteristics. Some basic concepts are introduced in this section. Others can be referred to G. Fan et al. (2013).

**Definition 4.** A 6-ary tuple $\Sigma = (N, IO, D, A_T, A_F, M_0)$ is called Primary Service Net (PSN), where:

1. $N = (P, T, F)$ is a basic Petri net, where $P$, $T$, $F$ denote the finite set of place, transition and arc.
2. $IO \subset P$ is a particular set called the $\Sigma$ interface.
3. $D$ is a non-empty finite individual set, and $f_D$ and $f_S$ are given as predicate sets and symbol sets, respectively.
4. $A_T$: $T \rightarrow f_D$, for $t \in T$, the free variable in $A_T(t)$ must be a directed arc free variable with $t$ as one end.
5. $A_F$: $F \rightarrow f_S$, if $(p, t) \in F$ or $(t, p) \in F$, then $A_F(p, t)$ or $A_F(t, p)$ is the sum of $n$-ary symbols and is null by default.
6. $M$: $P \rightarrow f_S$ is the marking of $\Sigma$, $p \in P$, initial marking $M_0(p)$ does not contain any free variables.

PSN is mainly used to model the execution process of available microservice instances. The state, possible change and the parameter of available microservice instances are described by place, transition and interface, respectively. For any $x \in (P \cup T)$, set ${}^\bullet x = \{y | y \in (P \cup T) \wedge (y, x) \in F\}$ and $x^\bullet = \{y | y \in (P \cup T) \wedge (x, y) \in F\}$ correspond to the input and output of $x$, respectively. For any $t_i \in T$, the input/output arc of transition $t_i$ and the set of free variables in $A_T(t_i)$ are denoted as $FV(t_i)$.

**Definition 5.** A 6-ary tuple $\Omega = (\Sigma, \Gamma, TI, TA, PI, PA)$ is called a Hierarchical Microservice Composition Net (HMCN).

1. $\Sigma$ is a PSN, which describes the basic structure of $\Omega$.
2. $\Gamma = \{\Gamma_i | i \in N^*\}$ is a finite set of pages, PSN or HMCN. If $N_i = (P_i, T_i, F_i)$, then $\forall \Gamma_i, \Gamma_j \in \Gamma$, $(P_i \cup T_i \cup F_i) \cap (P_j \cup T_j \cup F_j) = \emptyset$.
3. $TI \subset T$ is a collection of alternative nodes, where each page corresponds to a replacement node.
4. $TA$: $TI \rightarrow \Gamma$ is the page allocation function to assign specific pages for each replacement node.
5. $PI \subset P$ is a set of interface nodes that describe the substitute nodes' input and output locations.
6. $PA$ is a port mapping function. The purpose is to map the interface node of the replacement node to the interface of the corresponding page.

PSN is a particular case of HMCN, namely HMCN with $\Gamma$, which is equal to null. HMCN is mainly used for modelling components or the whole microservice composition process. Individual set $D$ is mainly used to describe microservice set or available service instance set. In this article, microservice *ms* and microservice instance *msi* are abstracted into individuals. Other individuals are abstracted as $\phi$ in HMCN.

In order to distinguish symbols, interface nodes and optional transitions are marked with upper case letters, and other places and transitions are marked with lower case letters. Besides, the input and output interface are symbolized with superscript $I$ and superscript $O$, respectively.

**Definition 6.** An aspect-oriented microservice net is a pair $AOM = (BN, AN)$, where:

1. $BN$ is a set of basic nets, and HMCN describes the core concerns of the system.

14680394, 2022, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.12924 by Ned Univ of Engineering & Tech, Wiley Online Library on [08/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

2. $AN = (AN_1, AN_2, ..., AN_n)$ is a collection of aspect nets, which describes the crosscutting concerns of the system. Aspect net is defined as a triple tuple $AN_i = (CutN, CN, C_t)$, in which $CutN:\{N_i \bullet x_j, ..., N_f \bullet x_k\}$ is called pointcut and $N_i \bullet x_j, ..., N_f \bullet x_k$ are join points of pointcut, advice net $CN = (PSN, t_{advice})$ and weaving type $C_t = \{before, after, around\}$.

The before and after types of aspect nets weave into the basic net before and after the entry point, respectively, while the around type aspect nets use the 'inhibition arc' to prevent aspect nets replace the transition pointed by the pointcut from firing the corresponding transitions. Advice net describes the implementation logic and introduction process of the pointcut, respectively. The aspect net structure weaves the introduction net and advice net into the base net according to the weaving rules. It is assumed that all places in PSN or HMCN can only store individuality.

The reliability model integration of the whole microservice system includes the following steps: (1) Extracting the business process as the essential concern and creating the primary net of microservices and instances. At this stage, templates are mainly used to improve scalability. (2) Extracting reliability-related operations as crosscutting concerns and related aspects net is established. The rules of the corresponding aspect net are given. (3) According to the weaving rules, weaving the aspect net with the basic net to form a reliability model. (4) Merging the exact place and transition, setting the initial marking and prioritizing the transition. Set all nodes to the idle locations and all applications to wait for starting locations.

For $\forall t \in T$, if $FV(t_i) = \{x_1, x_2, ..., x_n\}$, the individuality set $\{d_1, d_2, ..., d_n\}$ meets $d_i \in \{M(p) | p \in \bullet t \cup t \bullet\}$ and $d_i$ corresponds to the variable $x_i$, denoted by $t <d_1, d_2, ..., d_n>$. $A_T(t) <d_1, d_2, ..., d_n>$ and $A_F(p, t) <d_1, d_2, ..., d_n>$ describes that the individuals $d_1, d_2, ..., d_n$ replace the formula $A_T(t)$ and the predicate $A_F(p, t)$, respectively. If $A_T(t) <d_1, d_2, ..., d_n> = true$, then $t <d_1, d_2, ..., d_n>$ is called a feasible replacement of transition $t$ under $M$. All feasible replacements of transition $t$ under $M$ are denoted by set $VP(M, t)$. If $VP(M, t) \neq \emptyset$, then transition $t$ is enabled under marking $M$, denoted by $M[t>$. Moreover, transition $t$ is enabled under $M$ if and only if there is a possible replacement under marking $M$. All the enabled transitions under marking $M$ are denoted by set $ET(M)$.

For transition $t_i \in ET(M)$, if there is no any transition $t_j \in ET(M)$, whose priority is higher than $t_i$, the firing of transition $t_i$ under marking $M$ is effective. All the effective firing transitions under marking $M$ are denoted by set $FT(M)$. The process that marking $M$ reaches a new marking $M'$ by firing a feasible replacement $t_i <d_1, d_2, ..., d_n>$ of transition $t_i$ is denoted by $M[t_i <d_1, d_2, ..., d_n>>M'$. The concurrent transitions under marking $M$ are denoted by $MT(M)$. $H(M) = \{t <d_1, d_2, ..., d_n> | t \in MT(M), t <d_1, d_2, ..., d_n> \in VP(M, t)\}$ is called the greatest concurrent set of marking $M$.

If there is a firing sequence $H_1, H_2, ....., H_k$ and marking sequence $M_1, M_2, ..., M_k$, which make $M[H_1 > M_1[H_2 > M_2, ....., M_{k-1}[H_k > M_k$, then $M_k$ is a reachable marking from $M$. All the possibly reachable markings of $M$ are denoted by $R(M)$ and $M \in R(M)$.

## 4.2 | Microservice redundancy

In microservice-based cloud applications, the dependencies determine the order of execution. Accordingly, the dependency between microservice $ms_u$ and microservice $ms_i$ is modelled by HMCN model in Figure 3. The place $p_{s(u)}$ and the place $p_{e(u)}$ represent the start and termination state of the microservice $ms_u$ execution. The transition $t_{d(u,j)}$ represents the operation of the dependency relationship. Since the microservice $ms_u$ has been executed, $M(p_{s(u,i)}) \neq \emptyset$ and the transition $t_{d(u,j)}$ will be fired. The places in set $\bullet t_{s(i)} = \{p_{s(i)}\}$ both occupy the token and the microservice $ms_i$ executes.

In order to ensure the reliability of cloud applications, microservice instances are redundant during execution. In modelling, multiple instances are transformed into pages of HMCN model. The substitute nodes $p_{s(u)}$ and $p_{e(u)}$ are mapped into the page $N_u$ of components ${P_{s(u)}}^I$ and ${P_{e(u)}}^O$. The places ${P_{s(u)}}^I$ and ${P_{e(u)}}^O$ are input and output interfaces in Figure 4.

The microservice redundancies are symbolized as token in the place $p_{msi(a)}$ or the place $p_{msi(ua)}$. The transition in the set ${P_{s(u)}}^{I\bullet} = \{t_{msi(ua)}, t_{msi(a)}\}$ represent the execution of microservice with *available* and *unavailable* states. Because microservices have multiple instances, so a precondition is given that the initial marking of HMCN model is $|M_0(N_u.p_{msi(a)})| + |M_0(N_u.p_{msi(ua)})| \geq 1$. The state of instance can change from available to unavailable, when the failure occurs on it. As shown in Figure 4,

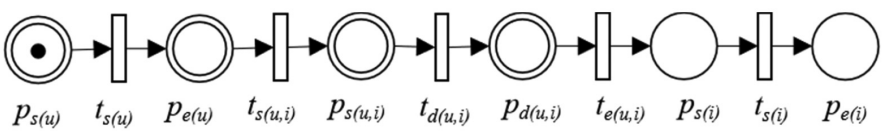



**FIGURE 3** Microservice modelling with a dependency relationship

14680394, 2022, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.12924 by Ned Univ of Engineering & Tech, Wiley Online Library on [08/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

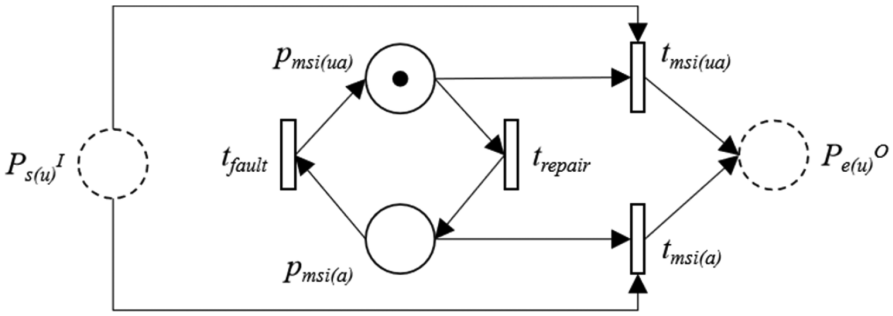



**FIGURE 4** Redundancy design with microservice instances on the page $N_u$

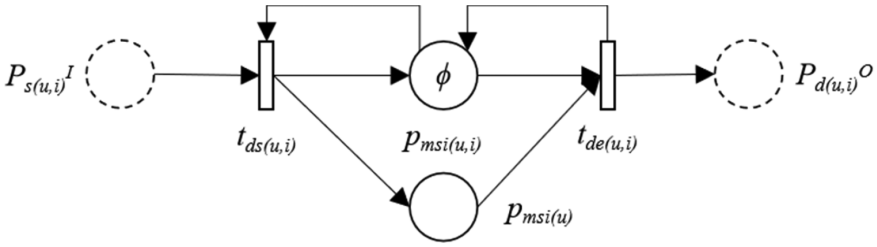



**FIGURE 5** Dependency design of microservices on the page $N_{(u,i)}$

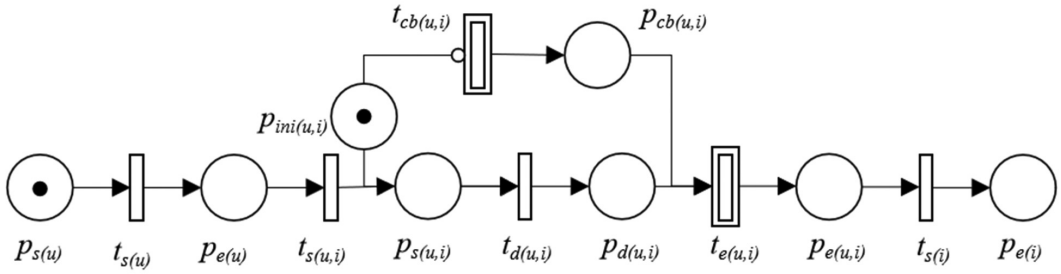



**FIGURE 6** Microservice modelling with circuit breaker integration

1. When the instance is unavailable ($M(N_u.p_{msi(ua)}) \neq \emptyset$), the transition $N_u.t_{repair}$ can fire to make instance redundancies return normal. Otherwise, when the instance is available ($M(N_1.p_{msi(a)}) \neq \emptyset$), the transition $N_u.t_{fault}$ in the page of HMCN model can fire and the instance redundancies becomes abnormal.
2. When the instance is available ($M(N_u.p_{msi(a)}) \neq \emptyset$) and the state of token in the place $P_{s(u)}{}^{I}$ is normal ($M(N_u.p_{msi(ua)}).s == normal$), the transition $N_u.t_{msi(a)}$ fires and the state of successor microservice is normal. When the instance is unavailable ($M(N_u.p_{msi(ua)}) \neq \emptyset$) or the state of token in the place $P_{s(u)}{}^{I}$ is abnormal ($M(N_u.p_{msi(ua)}).s == abnormal$), the transition $N_u.t_{msi(ua)}$ fires and the state of successor microservice is abnormal. In addition, the transition $N_u.t_{msi(a)}$ has higher priority than the transition $N_u.t_{msi(ua)}$.

The substitute nodes $p_{s(u,i)}$ and $p_{e(u,i)}$ are mapped into the page $N_{(u,i)}$ of components $P_{s(u,i)}{}^{I}$ and $P_{e(u,i)}{}^{O}$. The places $P_{s(u,i)}{}^{I}$ and $P_{e(u,i)}{}^{O}$ are input and output interfaces in Figure 5. The individual in the place $p_{msi(u,i)}$ is $\phi$, that is $M_0(p_{msi(u,i)}) = \phi$. Transition $t_{ds(u,i)}$ fires before the transition $t_{de(u,i)}$. This transition sequence illustrates the dependency relationship between $ms_u$ and $ms_i$.

## 4.3 | Circuit breaker design

The circuit breaker has been modelled as an aspect-oriented HMCN net, as shown. An inhibition arc has been added between the place $p_{ini(u,i)}$ and the transition $t_{cb(u,i)}$, as shown in Figure 6. According to Definition 6, the transition $t_{cb(u,i)}$ fires before the transition $t_{d(u,i)}$ and then the transition

14680394, 2022, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.12924 by Ned Univ of Engineering & Tech, Wiley Online Library on [08/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

$t_{e(u,i)}$. Based on the concept of aspect-oriented design, the circuit breaker concerns about the anti-cascade of failure without affecting dependency or redundancy design.

Details about the circuit breaker are modelled with page $N_{(u,i)}$ in HMCN. The substitute nodes $p_{e(u,i)}$ are mapped into page $N_{(u,i)}$ of components ${T_{cb(u,i)}}^I$ and ${T_{e(u,i)}}^O$. The transitions ${T_{cb(u,i)}}^I$ and ${T_{e(u,i)}}^O$ are input and output interfaces in Figure 7.

The circuit breaker has closed, open and semi-open states, which are the places $p_{closed}$, $p_{opened}$ and $p_{semi}$, respectively. Place $p_{ini}$ and $p_{term}$ are the initial and terminal state of the circuit breaker. The transitions illustrated below are the state transfer operation within the circuit breaker on the page $N_{(u,i)}$.

1. The transition $t_{stoc}$ represents the transfer operation between semi-open to closed. The predicated function of the transition $t_{stoc}$ is defined as $A_T(t_{stoc}) = (ms.s == normal)$.
2. The transition $t_{otos}$ represents the transfer operation between open to semi-open. The predicated function of the transition $t_{otos}$ is defined as $A_T(t_{otos}) = (t > T)$, where $BT$ is the block time of circuit breaker.
3. The transition $t_{stoo}$ represent the transfer operation between semi-open to open. The predicated function of the transition $t_{stoo}$ is defined as $A_T(t_{stoo}) = (ms.s == abnormal)$.
4. The transition $t_{ctos}$ represent the transfer operation between closed to open. The predicated function of the transition $t_{ctos}$ is defined as $A_T(t_{ctos}) = (n > N)$. Where $N$ is the maximal failure times of circuit breaker.

Besides that, for the microservice with normal state, the transition $t_{normal}$ fires and for the microservice with abnormal state, the transition $t_{abnormal}$ fires.

## 5 | RELIABILITY ANALYSIS OF MICROSERVICE DESIGN

### 5.1 | Correctness analysis of HMCN model

According to the syntax and semantics of the HMCN model, the state space of the model is limited, so the state space of the model can be used to verify the relevant properties. In this section, the model-based state-space analysis model can correctly describe the microservice cloud application business process, dynamic response redundancy and other processes. According to the reliable requirements of microservice cloud platform, starting with the modelling process and structure of the model, this article analyses the correctness of the execution of business process and instance redundancy.

**Property 1.** For the page $N_u$ of microservice $ms_u$ with redundancy design, if $M_0(N_u.P_{s(u)}{}^I) = ms$ and $M_0(N_u.P_{msi(ua)}) = msi$, then

1. $\exists M_1 \in R(M_0), M_1(N_u.P_{msi(a)}) = msi$.
2. $\exists M_1 \in R(M_0), (M_1(N_u.P_{msi(a)}) = \emptyset) \vee (M_1(N_u.P_{s(u)}{}^I).s == abnormal), \exists M_2 \in R(M_1), M_2(N_u.P_{e(u)}{}^O).s = abnormal$.
3. $\exists M_1 \in R(M_0), (M_1(N_u.P_{msi(a)}) \neq \emptyset) \wedge (M_1(N_u.P_{s(u)}{}^I).s == normal), \exists M_2 \in R(M_1), M_2(N_u.P_{e(u)}{}^O).s = normal$.

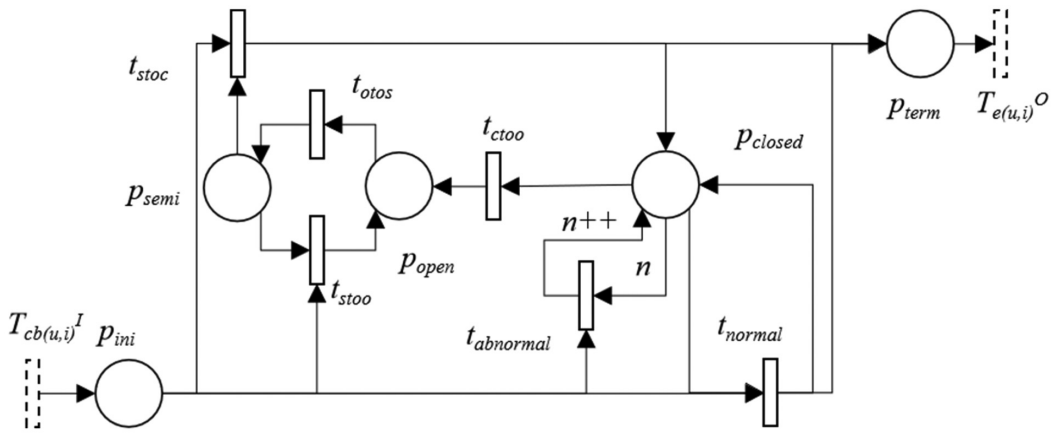



**FIGURE 7** Circuit breaker modelling with refinement on the page $N_{(u,i)}$

14680394, 2022, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.12924 by Ned Univ of Engineering & Tech, Wiley Online Library on [08/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

*Proof*. For Proposition (1), if msi.h = unavailable, the transition $t_{fault}$ fires. $\exists M_1 \in R(M_0)$, $M_1(N_u.P_{msi(a)}) = msi$. That means $(M_0(N_u.P_{msi(ua)}) \cup M_0(N_u.P_{msi(a)})) \neq \emptyset$.

If the condition $M_1(N_u.P_{s(u)}{}^I).s == abnormal$ is true, the transition $t_{msi(ua)}$ fires. $\exists M_2 \in R(M_1)$, $M_2(N_u.P_{e(u)}{}^O).s = abnormal$. In case of $M_1(N_u.P_{msi(a)}) = \emptyset$, derived from proposition (1), $M_1(N_u.P_{msi(ua)}).h = unavailable$ and the transition $t_{msi(ua)}$ fires. So, $\exists M_2 \in R(M_1)$, $M_2(N_u.P_{e(u)}{}^O).s = abnormal$ and Proposition (2) has been proofed.

The Proposition (3) can be proofed according to two situations: If $M_1(N_u.P_{msi(ua)}) = \emptyset$ and $M_1(N_u.P_{msi(a)}) \neq \emptyset$. In this case $M_1(N_u.P_{s(u)}{}^I).s == normal$, the transition $t_{msi(a)}$ fires, then $\exists M_2 \in R(M_1)$, $M_2(N_u.P_{e(u)}{}^O).s = normal$. If $M_1(N_u.P_{msi(ua)}) = msi'$ and $M_1(N_u.P_{msi(a)}) \neq \emptyset$, the transition $t_{msi(a)}$ fires, then $\exists M_2 \in R(M_1)$, $M_2(N_u.P_{e(u)}{}^O).s = normal$. Proposition (3) has been proofed as well.

Property 1 describes the state transfer of the microservice instances. Additionally, the predecessor and the microservice determinates the state with the HMCN model.

**Property 2.** For the HMCN model of microservice-based cloud application with dependency relationship, $\forall ms_i$ and $ms_j$, $(ms_i \in PRED(ms_u)) \wedge (ms_j \in PRED(ms_u))$, if $(\exists msi_{i,p}, msi_{j,q}, msi_{i,p} \in DE(ms_i), msi_{j,q} \in DE(ms_j)$ and $msi_{i,p}.h = msi_{j,q}.h = available) \wedge (\exists! ms_{u,r}, msi_{u,r} \in DE(ms_u)$ and $ms_{u,r}.h = unavailable)$, then $\exists! x \in \{p_{e(i)}, p_{e(j)}\}$, $\forall M \in R(M_0)$, $ms \in M(x)$ and $ms.s = abnormal$.

*Proof*. This proposition is proven by contradiction.

Assuming that $\exists M_1, M_2 \in R(M_0)$, $M_1(p_{e(i)}) = ms_i$ and $M_2(p_{e(j)}) = ms_j$, since $\exists msi_{i,p} \in DE(ms_i)$ and $\exists msi_{j,q} \in DE(ms_j)$, $msi_{i,p}.h = msi_{j,q}.h = available$ and the priority of the transition $N_i.t_{msi(a)}$ and the transition $N_i.t_{msi(a)}$ is higher than the transition $N_i.t_{msi(ua)}$ and the transition $N_j.t_{msi(ua)}$, respectively, $\exists M_3 \in R(M_1)$ and $\exists M_4 \in R(M_2)$, $(N_i.t_{msi(ua)} \in FT(M_3)) \wedge (N_j.t_{msi(ua)} \in FT(M_4))$ and the proposition $P1 = (M_3(p_{s(i)}).s = abnormal) \wedge (M_4(p_{s(j)}).s = abnormal)$.

Since $\exists!\ ms_{u,r} \in DE(ms_u)$ and $ms_{u,r}.h = unavailable$ and the priority of the transition $N_u.t_{msi(a)}$ is higher than the transition $N_u.t_{msi(ua)}$, $\exists M_5 \in R(M_3)$ and $\exists M_6 \in R(M_4)$, the proposition $P2 = (M_5(p_{s(i)}).s = normal) \vee (M_6(p_{s(j)}).s = normal)$.

Because $P1 = \neg P2$, thus the assumption is wrong.

Property 2 shows that the description model can correctly describe the impact of predecessor microservice by its successors. However, the failure of microservice $ms_u$ has only impact on one of its predecessors.

**Property 3.** For the HMCN model of microservice-based cloud application with circuit breaker integration, $\forall p \in \{p_{semi}, p_{closed}, p_{open}\}$, $\exists M_1 \in R(M_0)$ and $M_1(p) \neq \emptyset$.

*Proof*. This proposition is proven by proof of contradiction.

1. In case of $p = p_{semi}$, assuming that $\forall M_1 \in R(M_0)$, $M_1(p_{open}) = \emptyset$, so $t_{ctos} \notin FT(M_0)$ and $t_{stoo} \notin FT(M_0)$. If $ms.s = abnormal$ and $M_2(p_{semi}) \neq \emptyset$, $\exists M_2 \in R(M_0)$ and $\forall M_1 \in R(M_0)$, then $\exists M_2 \in R(M_1)$. $t_{stoo} \in FT(M_2)$. In another case. if $M_2(p_{closed}) \neq \emptyset$ and $N$ is a limited number, then $t_{ctos} \in FT(M_2)$.
2. In case of $p = p_{closed}$, assuming that $\forall M_1 \in R(M_0)$, $M_1(p_{closed}) = \emptyset$, so $t_{stoc} \notin FT(M_0)$ and $t_{abnormal} \notin FT(M_0)$. If $ms.s = normal$ and $M_2(p_{semi}) \neq \emptyset$, $\exists M_2 \in R(M_0)$ *and* $\forall\ M_1 \in R(M_0)$, then $\exists M_2 \in R(M_1)$. $t_{stoc} \in FT(M_2)$. If $ms.s = abnormal$, since $p_{closed} \in {}^\bullet t_{abnormal}$, then $t_{abnormal} \in FT(M_2)$.
3. In case of $p = p_{semi}$, assuming that $\forall M_1 \in R(M_0)$, $M_1(p_{semi}) = \emptyset$, so $t_{stos} \notin FT(M_0)$. Assumption (1) draws the conclusion, that $\exists M_2 \in R(M_1)$, $M_2(p_{open}) \neq \emptyset$. Because $p_{open} \in {}^\bullet t_{stos}$, after a period of time $T$, the transition $t_{stos}$ fires and then $M_1(p_{semi}) \neq \emptyset$.

The Assumptions (1), (2) and (3) are all wrong, hence the proposition is proofed to be true.

Property 3 shows that the HMCN model for circuit breaker modelling can reach three states of circuit breaker: closed, open and semi-open. Either of these states can be transferred from the other two states.

**Theorem 1.** For the HMCN modelled circuit breaker, if $\exists ms_i, ms_j \in PRED(ms_u)$ and $M_0(p_{s(u)}) = ms$, $\forall msi \in MSI(ms)$, msi.h = unavailable, $\exists M_1 \in R(M_0)$, $M_1(p_{s(i)}) = \emptyset$ or $M_1(p_{s(j)}) = \emptyset$.

14680394, 2022, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.12924 by Ned Univ of Engineering & Tech, Wiley Online Library on [08/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

*Proof*. From Property 1 $\exists M_1 \in R(M_0)$, $M_1(p_{e(u)}) = ms$ and $ms.s = abnormal$. The place $p_{s(u)}{}^{\bullet} = \{t_{s(u,i)}, t_{s(u,j)}\}$ and $M_0(p_{ini(u,i)}) = M_0(p_{ini(u,i)}) \neq \emptyset$, so one of the transitions $t_{s(u,i)}$ and $t_{s(u,j)}$ fires. According to the around type of AOM net, the transitions $t_{e(u,i)}$ and $t_{e(u,j)}$ fire after the transition sequence. Finally, it draws the conclusion that $\exists M_2 \in R(M_1)$, $(M_2(p_{s(i)}) == \emptyset) \vee (M_2(p_{s(j)}) == \emptyset) = \text{true}$.

Theorem 1 shows that the use of circuit breaker is mainly related to dependency.

## 5.2 | Strategy of microservice reliability

With the HMCN model, the strategies in response to the specified reliability problems are put forward. Based on the formal model HMCN execution semantics of microservice cloud applications, we propose a reliable composition strategy of microservice cloud applications. This strategy consists of two parts: redundancy strategy for the failure of a single microservice and circuit breaker for eliminating failure affects other dependent microservices. Through this composition strategy, the reliability of microservice cloud applications is improved.

### 5.2.1 | Redundancy strategy

In microservice based cloud application, each microservice has redundancy, which is used as an alternative to improve the reliability of microservices. The reliability of a microservice instance $msi_{i,j}$ can be calculated by the formula:

$$r_msi_{i,j} = e^{-\lambda_{i,j} * t} \tag{1}$$

The reliability of microservices $ms_i$ can be calculated by the formula:

$$r_ms_i = 1 - \prod_{j=1}^{|Re(ms_i)|} (1 - r_msi_{i,j}) \tag{2}$$

where $|Re(ms_i)|$ is the redundancy as alternative for microservices $ms_i$. How to select microservice instances as redundancy is a problem to be solved when the computational cost is limited, as shown below:

$$PR = max\, R_{ms} \tag{3}$$

where

$$R_{ms} = \prod_{i=1}^{|MS|} r_ms_i \tag{4}$$

s.t.

$$\sum_{i=1}^{|MS|} \sum_{j=1}^{|DE(ms_i) \cap PR|} c_{i,j} < C \tag{5}$$

where $|MS|$ is the number of microservices in the cloud application, $C$ is the limit of computing cost, $PR$ is the optimized reliability.

According to the relationships and properties of microservices and microservice instances, the redundancy strategy has been proposed.

1. For possible number of microservice instances of microservice, calculate the reliability through (Line 1–4 in Algorithm 1) and accumulate the cost (Line 5 in Algorithm 1).
2. Add the possible redundancy to the optimized reliability $PR$. If the reliability is less than others and cost is higher, then it is dominated. If the computing cost exceeds, the possible redundancy scheme is ignored (Line 7–8 in Algorithm 1).
3. Update the $PR$ for the next iteration of microservice and go back to Step 1 until all microservices in cloud application are done.

14680394, 2022, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.12924 by Ned Univ of Engineering & Tech, Wiley Online Library on [08/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

**Algorithm 1**

**Redundancy algorithm**

**Input**: microservice $ms$ with possible redundancy; failure rate and cost for each microservice instance; computing cost limit $C$.
**Output**: optimized reliability $PR$.

```
for microservice ms_i in cloud application do
  optimized reliability PRn = 0;
  for microservice instance ms_{i,j} of possible redundancy do
    calculate the reliability of microservice instance r_ms_{i,j} with Equation (1);
    accumulation the cost of microservice instance c_{i,j};
    add possible redundancy pr_{i,j} to PR;
    if (r_ms_{i,j} is less than others and c_{i,j} is higher) or (c_{i,j} is greater than C) then:
      remove possible redundancy pr_{i,j} from PR;
    end if
return PR;
```

This algorithm is to get the possible redundancy scheme through the method of dynamic linear programming. During the planning process, after the dominated scheme is ignored, the redundant strategy is optimized. The result is the redundancy scheme under computing resource constraints to achieve the optimization goal of maximum reliability.

### 5.2.2 | Circuit breaker strategy

According to the model design of HMCN net, the circuit breaker has two parameters named block time *BT* and sensitivity *K*. In this section, the parameters *BT* and *K* are calculated for microservice reliability, respectively. Due to the circuit breaker integration, when the state of the circuit breaker is open, the microservice needs to wait for *BT* before it can be requested. When the execution time of the microservice exceeds the deadline, the microservice is considered as failure. In another case, when failure trials exceed the threshold, the circuit breaker opens. The parameter *K* is used to measure the sensitivity of the circuit breaker.

In the microservice-based cloud application, each microservice owns the attribute deadline. Any microservice fails, then its impact on others in the cloud application will be analysed through availability. The guarantee ratio after failure is measured as criteria for the availability of microservices composition (Wang et al., 2015):

$$g_r = N_{\text{success}}/N_{\text{total}} \tag{6}$$

where $N_{\text{success}}$ means the number of microservice execution successfully and $N_{\text{total}}$ means the total number microservice.

In the process of practical application, where its reliability does not meet the requirements, a circuit breaker strategy for microservice-based cloud applications has been proposed to ensure the continuous and stable operation of the cloud application.

1. Initially, the microservices in the cloud application are allocated to container according to FESTAL algorithm (Wang et al., 2015). For each microservice $ms_i$:
   - In case of open state, $ms_i$ waits for *BT* and the state of circuit breaker changes to semi-open (Line 4–6 in Algorithm 2). The microservices in the cloud application are reallocated again because of waiting *BT*.
   - In case of semi-open state, if status of $ms_i$ is normal, then the state of circuit breaker changes to closed. If status of $ms_i$ is abnormal, then the state changes to open. (Line 4–6 in Algorithm 2).
   - In case of closed state, if status of $ms_i$ is normal, then the state of circuit breaker changes to closed. If status of $ms_i$ is abnormal and failure trials is less than thresholder, then the state stays closed, otherwise the state changes to open (Line 10–18 in Algorithm 2).
2. After allocation considering circuit breaker, the finish time of microservice $f_t_i$ is calculated (Line 20–21 in Algorithm 2). The guarantee ratio $g_r$ is calculated through the deadline $d_i$ and finish time of microservice $f_t_i$ (Line 22–25 in Algorithm 2).

14680394, 2022, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.12924 by Ned Univ of Engineering & Tech, Wiley Online Library on [08/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

**Algorithm 2**

**Circuit breaker algorithm**

**Input**: microservice set $MS$ in cloud application, microservice $ms_i$ in cloud application with deadline $d_i$, initialize $CB.state = closed$ and failure trials $n = 0$;
**Output**: guarantee ratio $gr$;

```
allocate all microservices in MS with FESTAL algorithm;
for each microservice ms_i in MS in cloud application do;
  switch CB.state:
    case open
      wait BT seconds, reallocate ms_i with FESTAL algorithm;
      CB.state = semi-open;
    case semi-open
      subcase 1: ms_i.s == normal, CB.state = closed;
      subcase 2: ms_i.s == abnormal, CB.state = open;
  case closed
    subcase 1: ms_i.s == normal, CB.state = closed;
    subcase 2: ms_i.s == abnormal;
      if n < N then
        CB.state = closed; n++
      end if
      if n == N then
        CB.state = open; n = 0
      end if
  for each microservice ms_i in MS in cloud application do;
    if ms_i.s == normal then f_t_i = s_t_i + e_t_i;
    if ms_i.s == abnormal then f_t_i = s_t_i + e_t_i + BT;
    if f_t_i D then N_succ++;
    N_total++;
  calculate g_r with Equation (6);
return g_r = N_succ/N_total;
```

According to the model design of HMCN net, the circuit breaker has three states: closed, open and semi-open. Google SRE overload protection algorithm is used to calculate the probability that the microservice request is discarded. The calculation is shown below:

$$\theta_{ij} = \max\{0, (request_{ij} - K * accepts_{ij})/(request_{ij} + 1)\} \tag{7}$$

where

$$request_{ij} = 1/freq_{ij} \tag{8}$$

$$accepts_{ij} = (1 - \lambda_j)/freq_{ij} \tag{9}$$

In Equation (7), the parameter *requests* is the number of microservice requests over time, *accepts* is the number of successful microservice requests, and *K* represents the magnification, the smaller *K* represents the more radical, and the smaller *K* represents the easier it is to discard the request. In Equations (8) and (9), the parameter $\lambda$ is the failure rate of microservice instance and the parameter *freq* is the request frequency between microservices.

The Equation (10) calculates the failure rate of the microservice dependency. Using this algorithm, microservice dependency does not directly discard all requests across the board, but calculates a probability to judge. When the number of successful requests is less and *K* is smaller, the

14680394, 2022, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.12924 by Ned Univ of Engineering & Tech, Wiley Online Library on [08/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

greater the value of *requests* − *K*accepts* and the greater the calculated probability, indicating the greater the probability that the request is discarded.

Based on the probability that the microservice request is discarded, the reliability of microservice dependency can be calculated by the formula:

$$r_dep_{i,j} = e^{-\theta_{i,j} * tran_{i,j}} \tag{10}$$

where $tran_{i,j}$ is the transition time between microservice $ms_i$ and microservice $ms_j$.

The following equation quantifies the reliability of microservice dependencies.

$$R_{dep} = \prod_{j=1}^{|MS|} \prod_{i=1}^{|MS|} r_dep_{i,j} \tag{11}$$

Where $r_dep_{i,j}$ is the reliability of microservice dependency. If there is a dependency relationship between microservice $ms_i$ and microservice $ms_j$, then is equal to 1, otherwise it is calculated by Equation (10).

Combined with Equation (4), the microservice based cloud application is calculated by the formula:

$$R = R_{ms} * R_{dep} \tag{12}$$

## 6 | CASE STUDY AND SIMULATION

The hospital information cloud application system is mainly composed of outpatient management, inpatient management, drug management, material management, equipment management, system management and other functional modules. The hospital cloud application system based on MSA achieves higher reliability, effectiveness and robustness (Hamzehloui et al., 2018). As the main function of outpatient management, the reservation management subsystem is composed of 11 microservices. The specific business process is shown in Figure 8.

A patient makes an appointment microservice $ms_0$ through access to the web-page microservice $ms_1$ firstly, which requests clinic diagnose microservice $ms_2$ and calendar microservice $ms_3$. Then the clinic diagnose microservice $ms_2$ depends on the doctor information microservice $ms_4$ and illness information microservice $ms_5$. According to the patient's condition and the doctor's specialty, patients are assigned to doctors of different specialties. After that, the patient needs to make an appointment online through the calendar. The calendar microservice $ms_3$ relies on the data from specialty database $ms_6$ and doctor database $ms_7$, respectively. Some illness catalogue, which is not stored in clinic database, can be searched with the external database. After accessing information microservice $ms_8$ and database $ms_9$, the patients can complete the clinic's appointment microservice $ms_{10}$. The proposed composition strategy is designed based on the structure to achieve that, the failure of microservice may not cause cascade effect.

By the HMCN net the scenario of the clinic appointment system has been modelled as the Figure 9 shows:

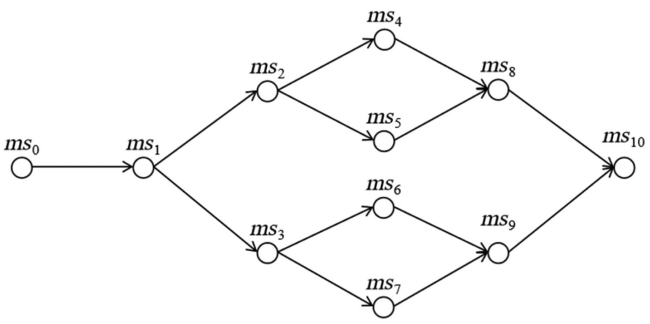



**FIGURE 8** Clinic appointment system with microservice-based cloud application

14680394, 2022, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.12924 by Ned Univ of Engineering & Tech, Wiley Online Library on [08/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

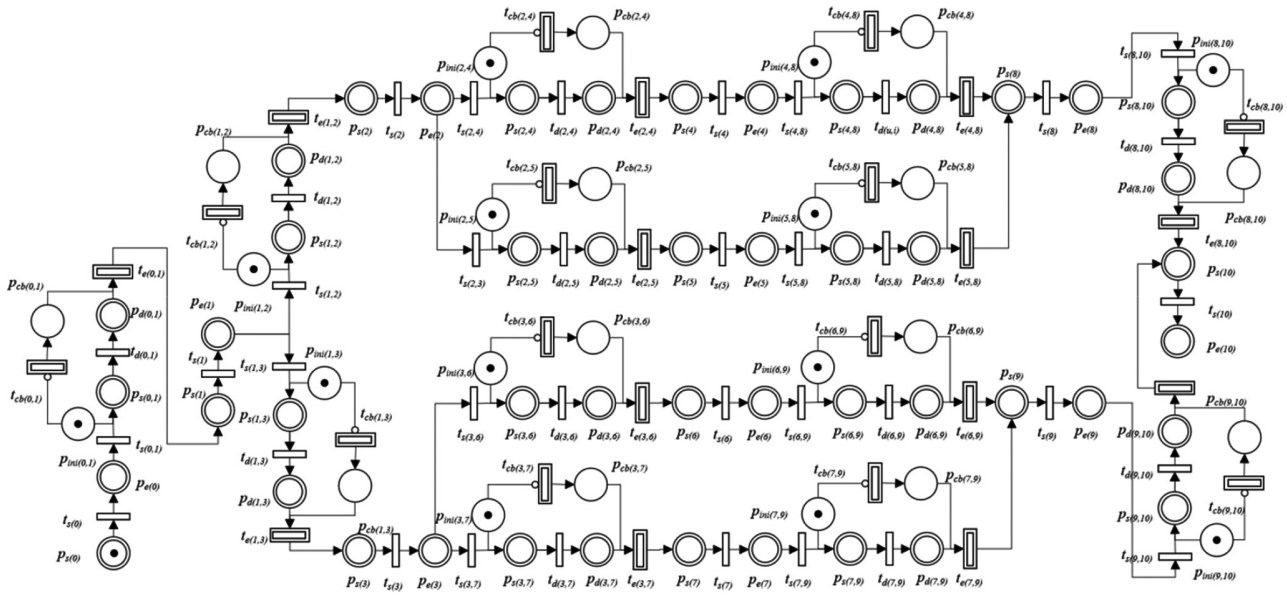


**FIGURE 9** Hierarchical Microservice Composition Net net model of the clinic appointment system

## 6.1 | Experiment with case study

In order to analyse the effectiveness of the reliability strategy proposed in this article, simulation experiments are designed to illustrate the effectiveness of the method. The reliability strategy is implemented in Java with Python 3.6.0 and PyCharm 2020.1.2. A program package for analysis and visualization of cloud application reliability is used to simulate microservice-based cloud applications.

This section analyses the effectiveness of the proposed composition strategy through experiments. The limited redundancy is allocated to each microservice. For the redundancy strategy of microservices, the experimental steps are as follows:

1. The cloud application of clinical system consists of 11 microservices. Initially, the failure rate of each microservice is randomly generated, and its value ranges from 0 to 0.002.
2. The 11 microservices have [2, 3, 2, 3, 2, 4, 2, 3, 2, 3, 1] redundancy as alternative. For example, the number of redundancies of the first microservice $ms_0$ is 2 and the number of redundancies of the second microservice $ms_1$ is 3 and so on.
3. According to the amount of computing resources, each redundancy has a different cost. In this experiment, the limit of computing cost is 7050 initially.
4. Through the redundancy strategy of microservice, the redundancy strategy meeting the computing cost is calculated, as shown in Figure 10.

It is not difficult to find from the Figure 10 that the green points represent dominated points. The optimized redundancy strategy is represented by the pareto frontier with red points.

Based on the case of clinical system, the strategy of circuit breaker is used to calculate the reliability of microservice dependency. The experimental steps are shown below:

1. Similarly, the failure rate of each microservice is randomly generated, and its value ranges from 0 to 0.002.
2. The request frequency is randomly generated with values from 1000 to 1200. Sensitivity $K$ ranges from 1.0002 to 1.001 every 0.0002. When setting parameters, it should be noted that the sensitivity of fuse should be less than the failure rate of microservice. Otherwise, the fuse will remain closed.
3. Through the circuit breaker strategy of microservice, the reliability of microservice dependency is calculated.

As shown in Figure 11, the reliability of microservice dependency depends on the request frequency and the dependent failure rate. In Figure 11, when the failure rate of the dependent microservices is lower than a certain value, the circuit breaker is always closed. Until the probability of error reaches the threshold, the circuit breaker opens and the dependent reliability is recalculated.

14680394, 2022, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.12924 by Ned Univ of Engineering & Tech, Wiley Online Library on [08/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

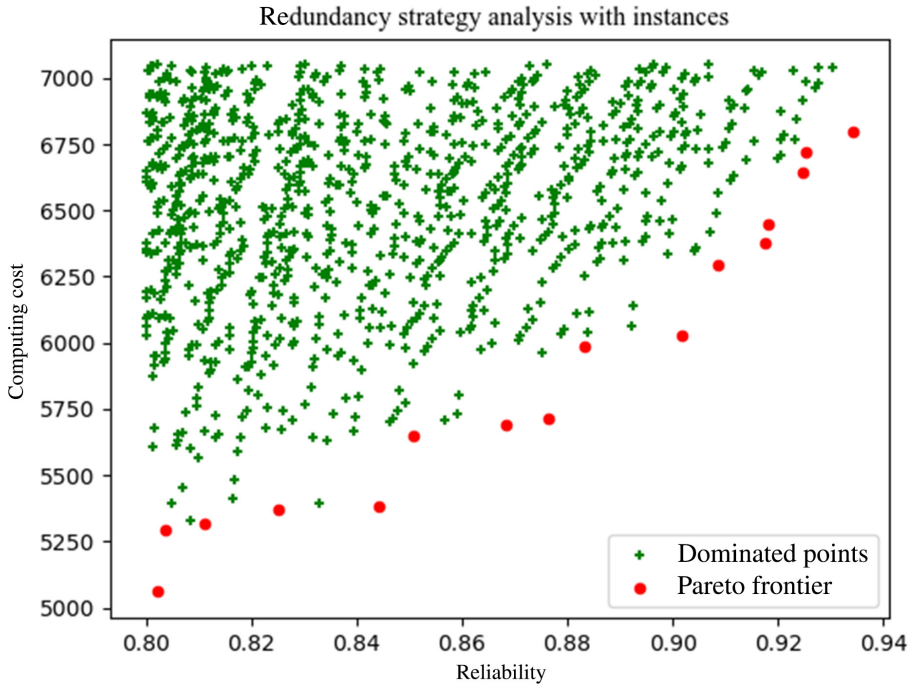



**FIGURE 10** Simulation result of clinic scenario with redundancy strategy

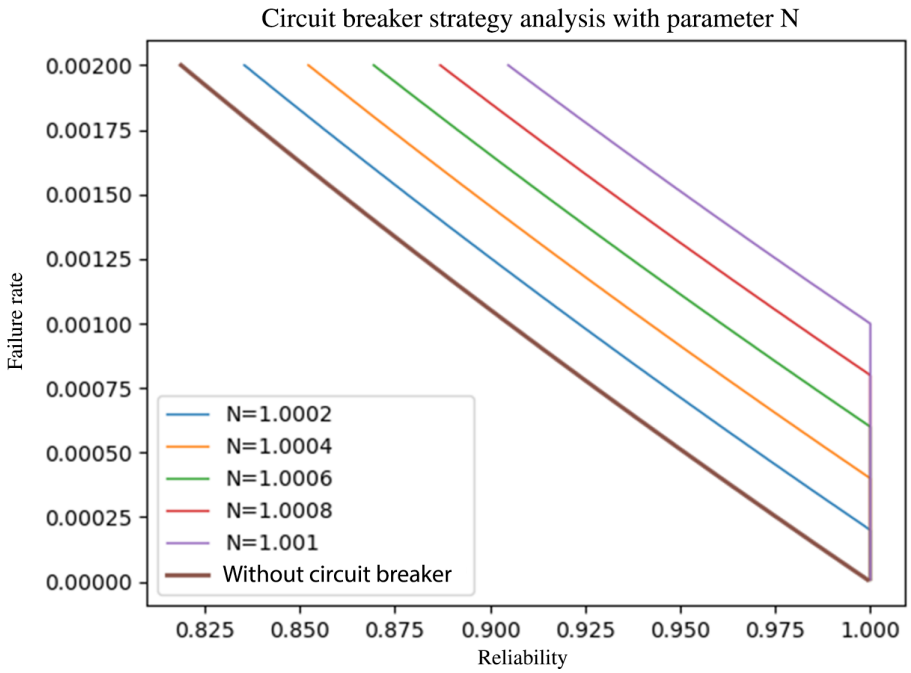



**FIGURE 11** Simulation result of clinic scenario with circuit breaker strategy

## 6.2 | Reliability simulation with complexity

To scientifically measure the improvement of fault tolerance effect of reliability improvement strategy, the simulation has done 50 groups of experiments and take the average value as result. Additionally, the block time *BT* was initially set to 15 ms in this experiment. This section discusses the effect of microservices complexity on reliability.

14680394, 2022, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.12924 by Ned Univ of Engineering & Tech, Wiley Online Library on [08/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

The experimental steps are demonstrated as follows:

1. Initially, the dependencies between them are randomly generated. The dependence degree is measured by $\beta \in [0,1]$. $\beta = 0$ means the microservices are independent, and $\beta = 1$ means these are full connections. In the experiment, the parameter $\beta$ is taken every 0.02 from 0.1 to 0.2.
2. In order to verify the effectiveness of the reliable strategy, we generate the experimental environment by randomly generating faults. The failure rate of microservices is randomly generated. For example, the failure rate of 0.2 means that 20% of the microservices in the microservice composition have failed.
3. Assume that the infrastructure of the cloud application contains four hosts with MIPS speed $[2000, 3000, 2000, 3000]$, the microservices are deployed in the container. The container has three types $[250, 500, 1000]$ with MIPS speed.
4. Using the redundancy and circuit breaker strategies, the percent of reliability improvement and guarantee ratio will be calculated.

The Figures 12–15 show the reliability simulation results with different complexity degrees applying strategies. From the simulation results, it can be seen that with the increase in the complexity of microservices, the reliability of cloud applications is declining. The impact of microservice failures on the whole application is strengthened, and they are not linear.

With the increasing complexity, the cascading effect of fault is noticeable, and the circuit breaker blocks the transmission of fault. Since the given host and container hardware devices are limited, microservice deployment may result in insufficient resources. Therefore, the redundancy strategy may increase resource load and decrease scheduling efficiency.

By observing the simulation results, it can be found that the effect of the redundancy strategy is not particularly obvious for the improvement of reliability. When doing replica deployment, it is required to consider resource scheduling and the microservice replicates consumes resources. However, the block time of the circuit breaker *BT* can be ignored, considering the impact on the schedule. Therefore, the fuse strategy is more effective for microservice with high complexity compared with the redundancy strategy. The circuit breaker can delay the negative effect of microservice caused by failure. The redundancy strategy can improve the reliability of microservice, especially at low complexity.

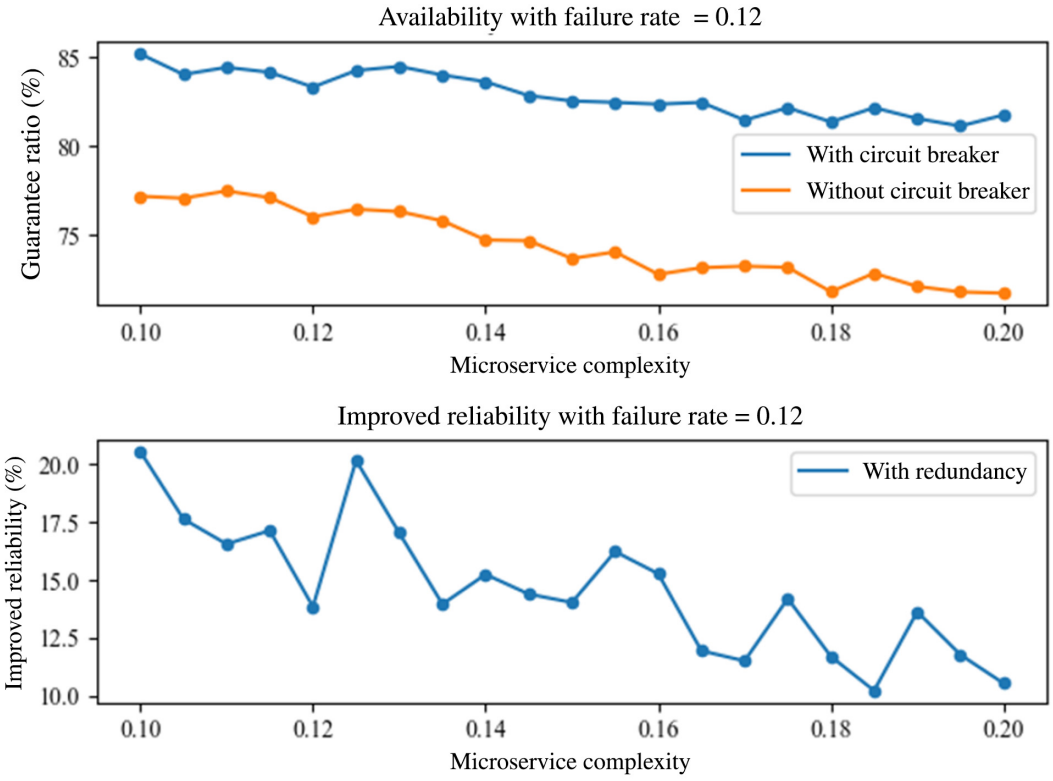



**FIGURE 12** Simulation results considering complexity beta with failure rate 0.12

14680394, 2022, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.12924 by Ned Univ of Engineering & Tech, Wiley Online Library on [08/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

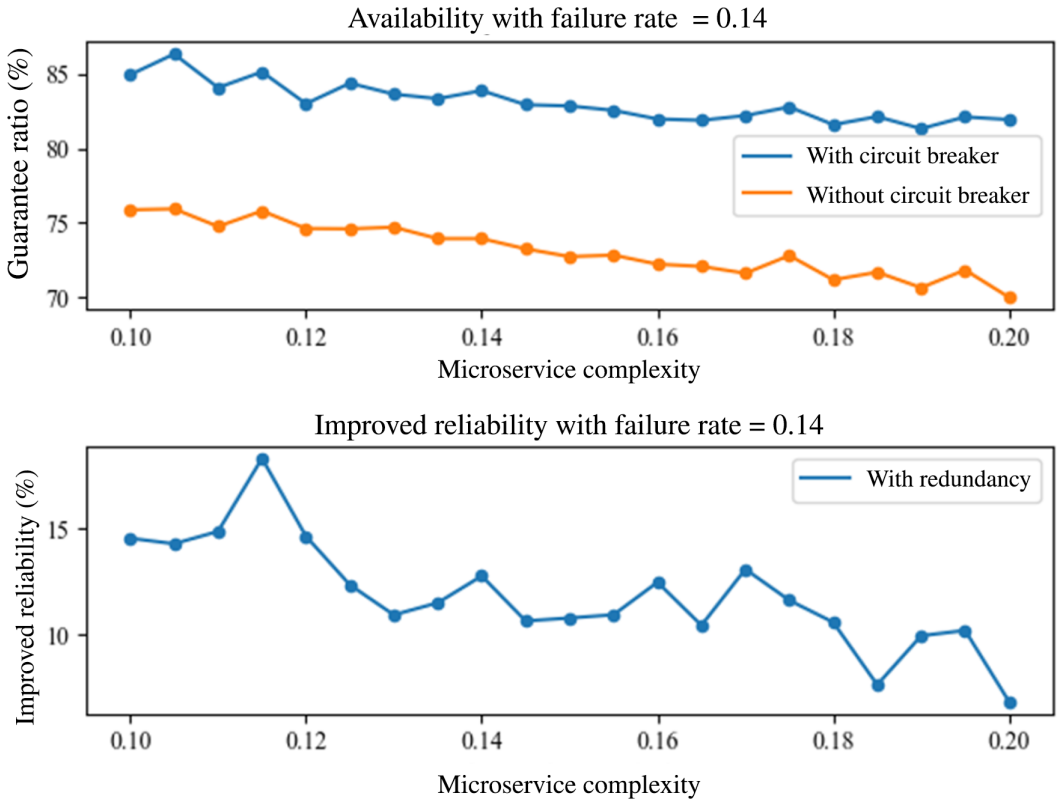



**FIGURE 13** Simulation results considering complexity beta with failure rate 0.14

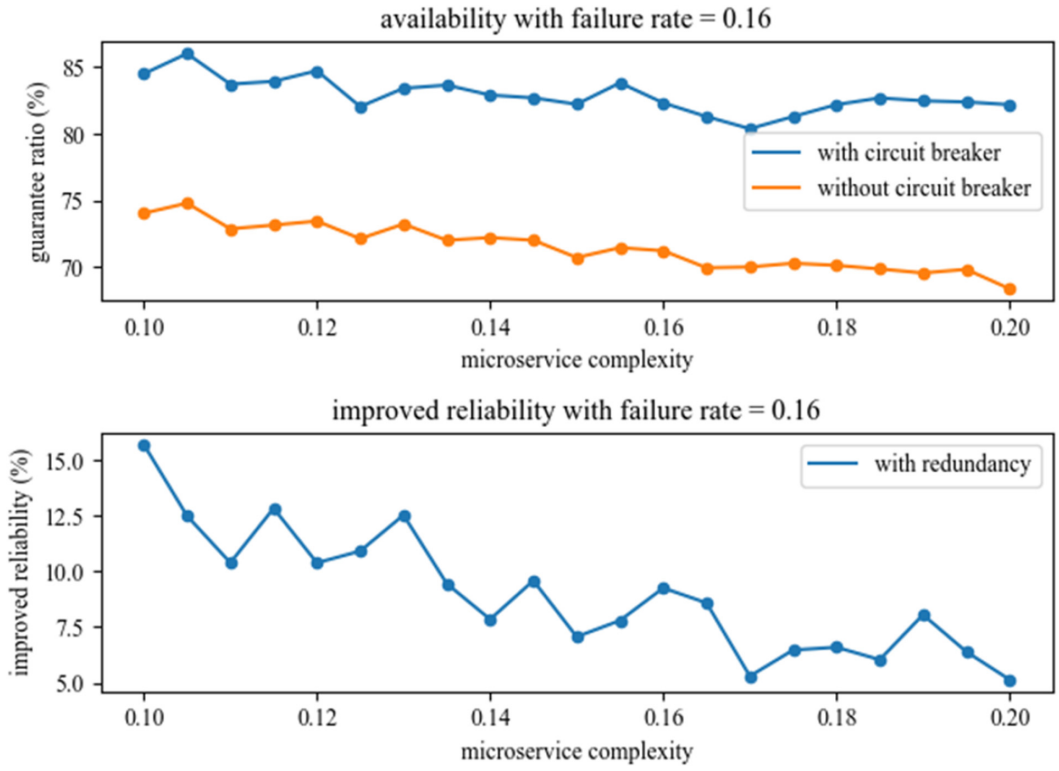



**FIGURE 14** Simulation results considering complexity beta with failure rate 0.16

14680394, 2022, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.12924 by Ned Univ of Engineering & Tech, Wiley Online Library on [08/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

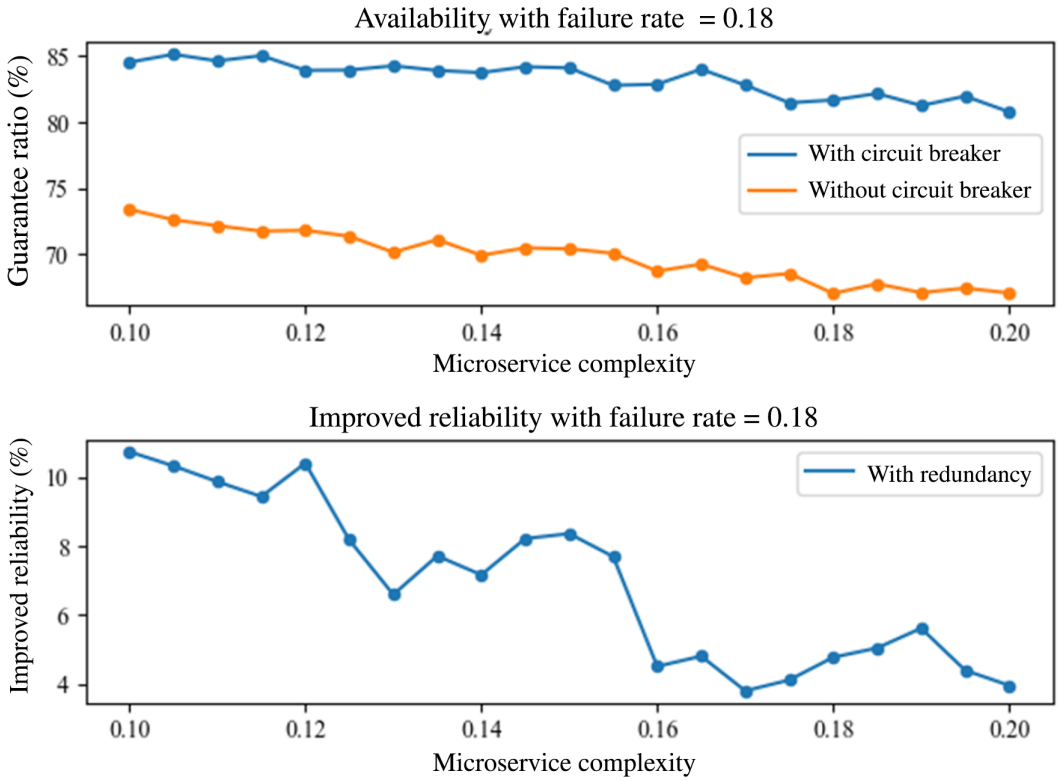



**FIGURE 15** Simulation results considering complexity beta with failure rate 0.18

## 6.3 | Reliability simulation with number

The impact of the number of microservices in the microservice composition has been discussed in the previous section in this section. In the experiment, it is assumed that $\beta = 0.2$ and the failure rate value is taken from 0.12 to 0.18. This section discusses the effect of microservices number of microservices on reliability.

The experimental steps are demonstrated as follows:

1. Initially the dependencies are initially generated with dependence degree similarly.
2. The failure rate is generated from 0.12 to 0.18 every 0.02. The task number is totally 100.
3. The infrastructure of the cloud application is setup like Section 6.2.
4. The percent of reliability improvement and guarantee ratio are calculated with composition strategy.

The reliability simulation results considering microservice count can be shown as Figures 16–19.

Reliability simulation results with different microservice numbers applying strategies. Similarly, when the failure rate is low, the effect of the circuit breaker is not obvious. With the increase of failure rate, the cascading effect of failure increases, and the circuit breaker plays a dominant role in improving reliability. In addition, as the number of microservices increases, the effect of the circuit breaker becomes more obvious. The redundancy strategy does not affect the improvement of microservice reliability with the increase in the number of microservices.

It can be found that when that, adding a circuit breaker will reduce the reliability. This is because the cascading failure caused by no circuit breaker results in the more available resource. These spare resources are scheduled by more microservices. That is, the improvement of scheduling efficiency makes up for the loss caused by the failure. Moreover, when the value of the failure rate increases, the negative effect of cascading failures plays a decisive role.

To conclude, the experimental reliability analysis procedure is given to illustrate the proposed technique, which shows an excellent fit for our goal.

14680394, 2022, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.12924 by Ned Univ of Engineering & Tech, Wiley Online Library on [08/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

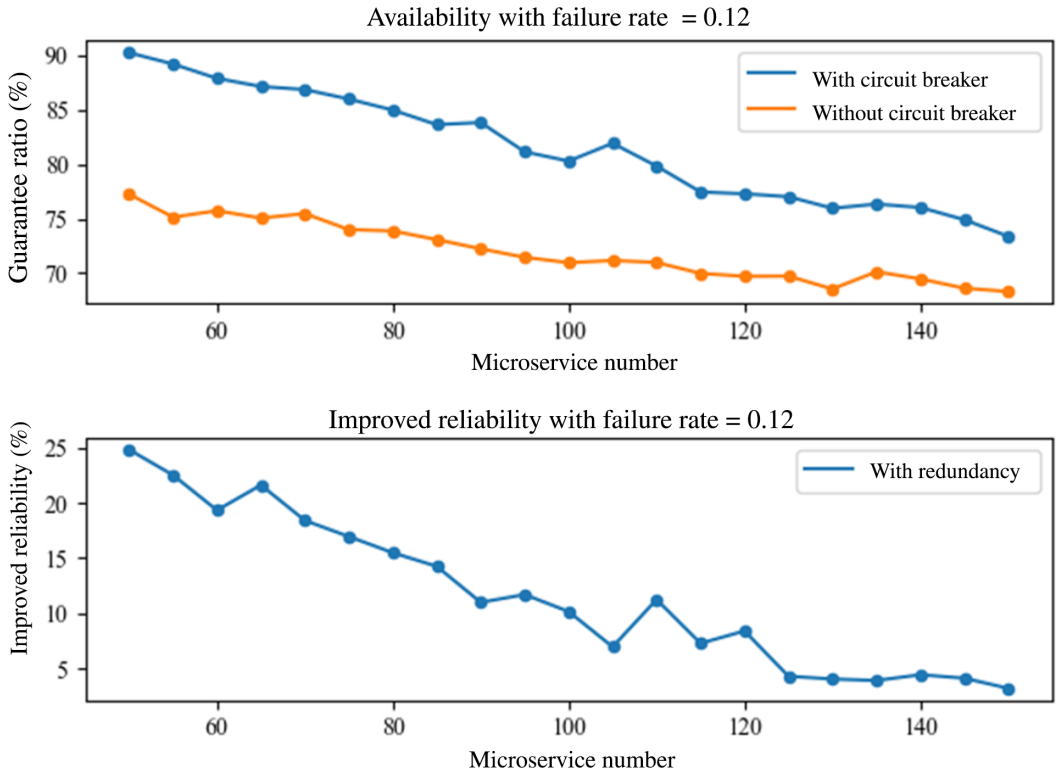



**FIGURE 16** Simulation results considering microservice number with failure rate 0.12

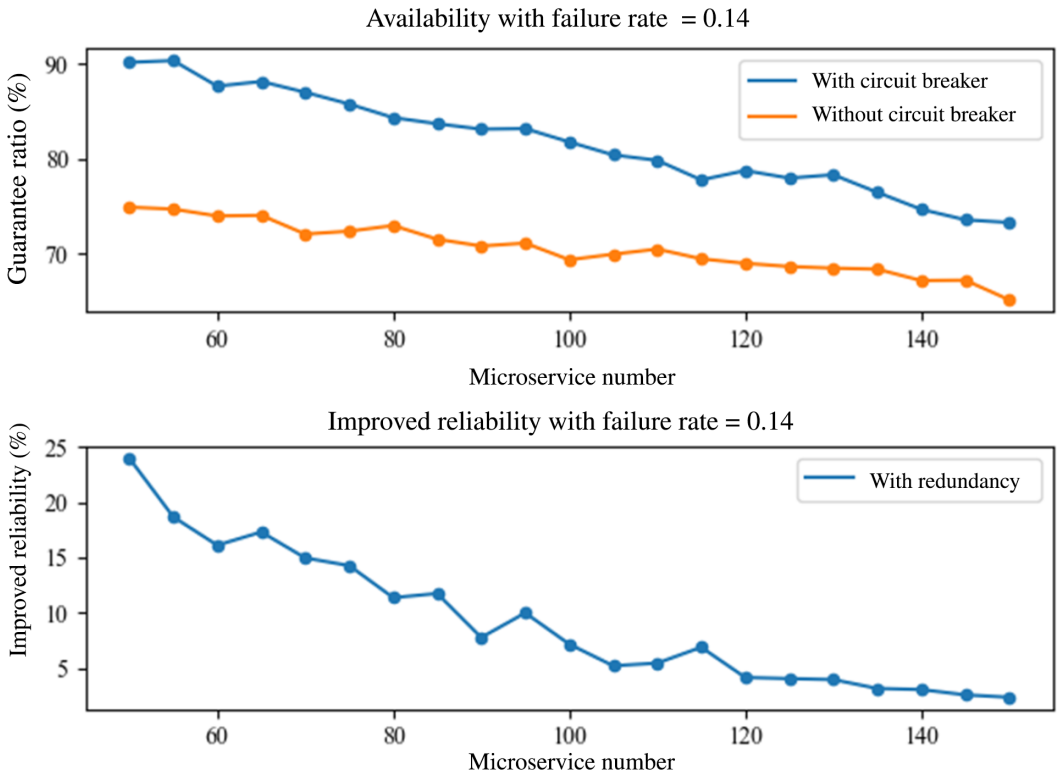



**FIGURE 17** Simulation results considering microservice number with failure rate 0.14

14680394, 2022, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.12924 by Ned Univ of Engineering & Tech, Wiley Online Library on [08/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

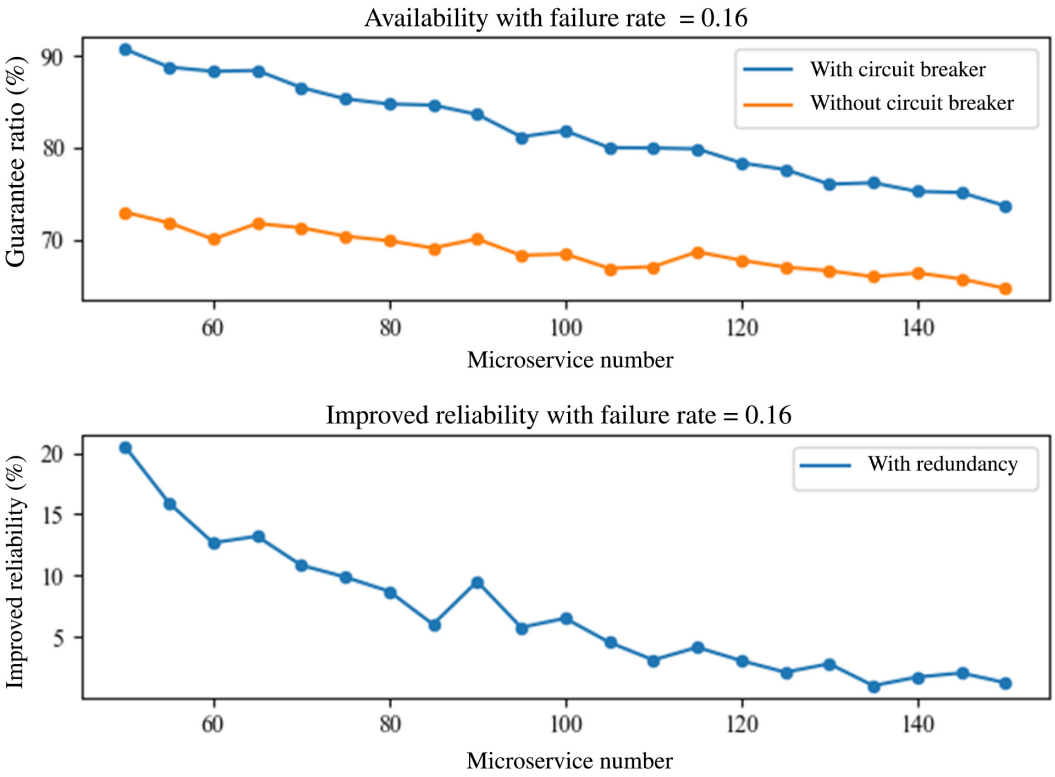



**FIGURE 18** Simulation results considering microservice number with failure rate 0.16

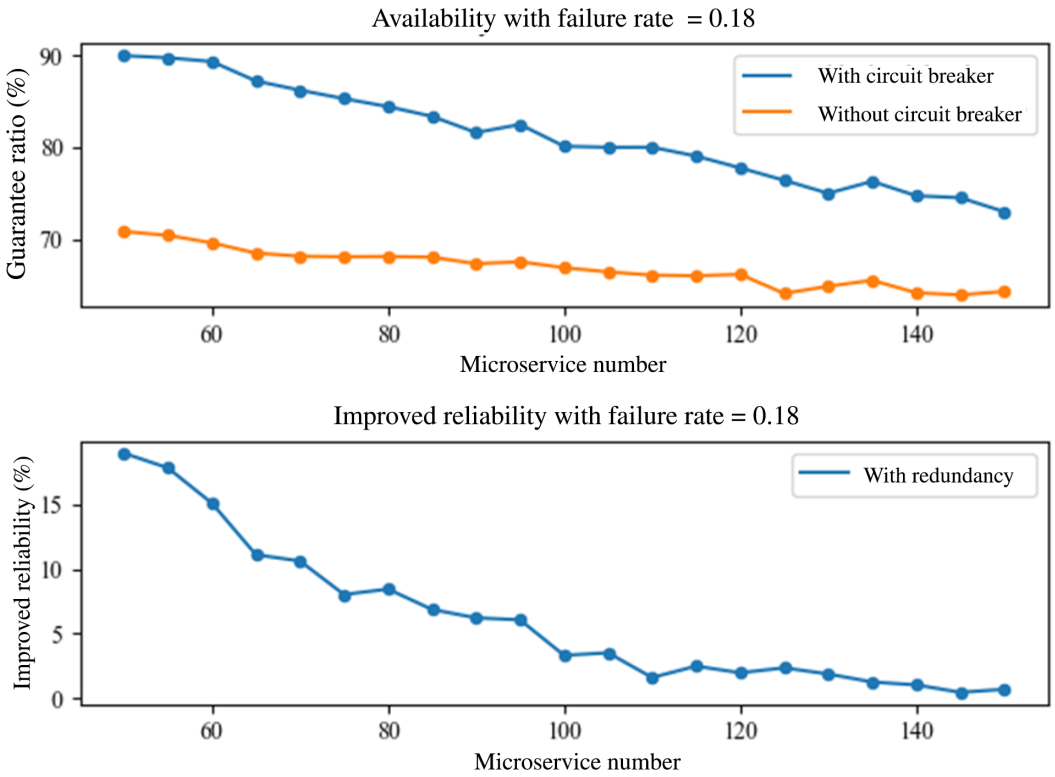



**FIGURE 19** Simulation results considering microservice number with failure rate 0.18

14680394, 2022, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.12924 by Ned Univ of Engineering & Tech, Wiley Online Library on [08/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

## 7 | CONCLUSION

Microservice design is increasingly popular due to its fine-grained method and loosely coupled services. The application designed and developed with microservice has better scalability and provides developers with the flexibility of expansion at the minimum cost. By applying MSA, microservices deploy these small modules by evolving the original monomer architecture into a smaller and more easily changed model.

1. The reliability modelling utilizes the concept of divide and conquers to decentralize multiple modules and data. Each service is deployed with a shared or isolated operation process. The reliability can be obtained by distributed risk. The microservice reliability model is analysed using the PrT net and verified through a formal method.
2. The redundancy strategy focuses on a single service reliability tactic. This article investigates the impact of adding redundancy and the availability of microservice-based applications. The numerical results based on the simulation data set verify that the performance of multiple microservice instances is significantly better than the benchmark.
3. The circuit breaker provides a microservice with essential functions such as call timeout, failure threshold and reliability calculation. The PrT net graphical tool represents the flexible rolling statistics, resource control, and controls over the states and the transitions of circuit breakers.

The composition strategy benefits reliability improvement under different circumstances and impacts each other. The time constraint of microservice will be further discussed in future work, and the energy consumption is the critical research point.

### CONFLICT OF INTEREST

The authors declare no potential conflict of interest.

### DATA AVAILABILITY STATEMENT

The data used to support the findings of this study are included within the article.

### ORCID

*Zheng Liu* https://orcid.org/0000-0002-3390-7894
*Guisheng Fan* https://orcid.org/0000-0002-2702-0242
*Huiqun Yu* https://orcid.org/0000-0002-1899-1135
*Liqiong Chen* https://orcid.org/0000-0003-1927-2148

### REFERENCES

Aderaldo, C. M., Mendonça, N. C., Pahl, C., & Jamshidi, P. (2017). Benchmark Requirements for Microservices Architecture Research. 2017 IEEE/ACM 1st International Workshop on Establishing the Community-Wide Infrastructure for Architecture-Based Software Engineering (ECASE), Buenos Aires, Argentina. https://doi.org/10.1109/ECASE.2017.4

Anne, K., Danilo, A., & Raffaela, M. (2013). Hybrid multi-attribute QoS optimization in component based software systems. *Journal of Systems and Software*, *86*(10), 2542–2558. https://doi.org/10.1016/j.jss.2013.03.081

Camilli, M. (2020). Continuous formal verification of microservice-based process flows. Software Architecture, 14th European Conference, ECSA 2020 Tracks and Workshops, L'Aquila, Italy, pp. 420–435. http://doi.org/10.1007/978-3-030-59155-7_31

Chaudhry, S. R., Palade, A., Kazmi, A., & Clarke, S. (2020). Improved QoS at the edge using Serverless computing to deploy virtual network functions. *IEEE Internet of Things Journal*, *7*(10), 10673–10683. https://doi.org/10.1109/JIOT.2020.3011057

Dai, F., Mo, Q., Qiang, Z., Huang, B., Kou, W., & Yang, H. (2020). A choreography analysis approach for microservice composition in cyber-physical-social systems. *IEEE Access*, *8*, 53215–53222. https://doi.org/10.1109/ACCESS.2020.2980891

Fan, G., Yu, H., & Chen, L. (2016). A formal aspect-oriented method for modeling and analyzing adaptive resource scheduling in cloud computing. *IEEE Transactions on Network and Service Management*, *13*(2), 281–294. https://doi.org/10.1109/TNSM.2016.2553157

Fan, G., Yu, H., Chen, L., & Liu, D. (2013). Modeling and optimizing resource scheduling for service composition based on queuing Petri Net. 2013 IEEE 37th Annual Computer Software and Applications Conference, Kyoto, Japan, pp. 529–538. https://doi.org/10.1109/COMPSAC.2013.87

Fokaefs, M., Barna, C., & Litoiu, M. (2016). Economics-driven resource scalability on the cloud. 2016 IEEE/ACM 11th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS), Austin, TX, pp. 129–139. https://doi.org/10.1109/SEAMS.2016.023

Gao, M., Chen, M., Liu, A., Hung, W., & Yung, K. L. (2020). Optimization of microservice composition based on artificial immune algorithm considering fuzziness and user preference. *IEEE Access*, *8*, 26385–26404. https://doi.org/10.1109/ACCESS.2020.2971379

Gu, H., Li, X., Liu, M., & Wang, S. (2021). Scheduling method with adaptive learning for microservice workflows with hybrid resource provisioning. *International Journal of Machine Learning and Cybernetics*, *12*, 3037–3048.

Hamzehloui, M. S., Sahibuddin, S., & Salah, K. (2018). A Systematic Mapping Study on Microservices. In: F. Saeed, N. Gazem, F. Mohammed, & A. Busalim (eds) *Recent Trends in Data Science and Soft Computing. IRICT 2018. Advances in Intelligent Systems and Computing*, vol 843. Springer, Cham. https://doi.org/10.1007/978-3-319-99007-1_100

Hasselbring, W., & Steinacker, G. (2017). Microservice architectures for scalability, agility and reliability in E-commerce. 2017 IEEE International Conference on Software A Conference, Gothenburg, Sweden. http://doi.org/10.1109/ICSAW.2017.11

14680394, 2022, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.12924 by Ned Univ of Engineering & Tech, Wiley Online Library on [08/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Hussain, W., Sohaib, O., Naderpour, M., & Gao, H. (2020). Cloud marginal resource allocation: A decision support model. *Mobile Networks and Applications*, *22*(25), 1418–1433. https://doi.org/10.1007/s11036-019-01457-7

Komarasamy, D., & Muthuswamy, V. (2015). Adaptive deadline based dependent job scheduling algorithm in cloud computing. 2015 Seventh International Conference on Advanced Computing (ICoAC), Chennai, India, pp. 1–5. https://doi.org/10.1109/ICoAC.2015.7562794

Li, W., & Guo, W. (2015). QoS prediction for dynamic reconfiguration of component based software system. *Journal of Systems and Software*, *102*(1), 12–34. https://doi.org/10.1016/j.jss.2014.12.001

Lyu, Z., Wei, H., Bai, X., & Lian, C. (2020). Microservice-based architecture for an energy management system. *IEEE Systems Journal*, *14*(4), 5061–5072. https://doi.org/10.1109/JSYST.2020.2981095

Mario, B., Saverio, G., Jacopo, M., & Lacopo, T. (2020). A formal approach to microservice architecture deployment. Microservices, Science and Engineering. http://doi.org/10.1007/978-3-030-31646-4_8

Niko, M., Timo, A., Mikko, R., Aleksandr, O., Sergey, A., Yevgeni, K., & Tommi, M. (2019). Action-oriented programming model: Collective executions and interactions in the fog. *Journal of Systems and Software*, *157*(1), 110391.

Noor, A., Jha, D. N., Mitra, K., Jayaraman, P. P., Souza, A., Ranjan, R., & Dustdar, S. (2019). A Framework for monitoring microservice-oriented cloud applications in heterogeneous virtualization environments. 2019 IEEE 12th International Conference on Cloud Computing (CLOUD), Milan, Italy, pp. 156–163. https://doi.org/10.1109/CLOUD.2019.00035

Petrasch, R. (2017, September 7). Model-based engineering for microservice architectures using Enterprise Integration Patterns for inter-service communication. 2017 14th International Joint Conference on Computer Science and Software Engineering (JCSSE), NakhonSiThammarat, Thailand, pp. 1–4. http://doi.org/10.1109/JCSSE.2017.8025912.

Pietrantuono, R., Russo, S., & Guerriero, A. (2018). Run-time reliability estimation of microservice architectures. 2018 IEEE 29th International Symposium on Software Reliability Engineering (ISSRE), Memphis, TN, pp. 25–35. https://doi.org/10.1109/ISSRE.2018.00014

Villamizar, M., Garcés, O., Ochoa, L., Castro, H., Salamanca, L., Verano, M., Casallas, R., Gil, S., Valencia, C., Zambrano, A., & Lang, M. (2016). Infrastructure cost comparison of running web applications in the cloud using AWS lambda and monolithic and microservice architectures. 2016 16th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid), Cartagena, Colombia, pp. 179–182. https://doi.org/10.1109/CCGrid.2016.37

Wang, J., Bao, W., Zhu, X., Yang, L. T., & Xiang, Y. (2015). FESTAL: Fault-tolerant elastic scheduling algorithm for real-time tasks in virtualized clouds. *IEEE Transactions on Computers*, *64*(9), 2545–2558. https://doi.org/10.1109/TC.2014.2366751

Wang, T., Zhang, W., Xu, J., & Gu, Z. (2020). Workflow-aware automatic fault diagnosis for microservice-based applications with statistics. *IEEE Transactions on Network and Service Management*, *17*(4), 2350–2363. https://doi.org/10.1109/TNSM.2020.3022028

Wang, Y., Cheng, L., & Sun, X. (2019). Design and research of microservice application automation testing framework. 2019 International Conference on Information Technology and Computer Application (ITCA), Guangzhou, China, pp. 257–260. https://doi.org/10.1109/ITCA49981.2019.00063

Wu, L., Tordsson, J., Elmroth, E., & Kao, O. (2020). MicroRCA: Root cause localization of performance issues in microservices. NOMS 2020–2020 IEEE/IFIP Network Operations and Management Symposium, Budapest, Hungary, pp. 1–9. https://doi.org/10.1109/NOMS47738.2020.9110353

Wu, Q., Ishikawa, F., Zhu, Q., Xia, Y., & Wen, J. (2018). Deadline-constrained cost optimization approaches for workflow scheduling in clouds. *IEEE Transactions on Parallel and Distributed Systems*, *28*(12), 3401–3412. https://doi.org/10.1109/TPDS.2017.2735400

Zhou, X., Peng, X., Xie, T., Sun, J., Ji, C., Li, W., & Ding, D. (2019). Delta debugging microservice systems with parallel optimization. *IEEE Transactions on Services Computing*, 1–14. https://doi.org/10.1109/TSC.2019.2919823

## AUTHOR BIOGRAPHIES

**Liu Zheng** is a PhD candidate at School of information science and engineering, East China University of technology. He received a bachelor's degree from Tongji University in 2011 and a master's degree from Technical University of Munich in 2014. His main research fields are micro service reliability, formal methods and cloud computing. He has published 3 papers in academic journals and conferences.

**Fan Guisheng** is currently an associate professor and master supervisor of the school of information science and engineering of East China University of technology, and a member of the special committee of software engineering of China computer society. In 2009, he received a doctor's degree in computer application technology from East China University of technology. From 2013 to 2014, he visited the school of computer science and engineering at the University of California, Davis. The main research fields are software engineering, trusted computing, formal methods and cloud computing. He presided over the research of projects such as the National Natural Science Foundation of China, the basic scientific research business expenses of Central Universities and the open topics of key laboratories of software engineering, published more than 60 papers in academic journals at home and abroad, and won the first prize of Shanghai Scientific and technological progress and the second prize of Shanghai industry university research.

**Yu Huiqun** is a professor, doctoral supervisor, senior member of IEEE, ACM member, senior member of China computer society, vice president of Shanghai microcomputer application society, executive director of Shanghai Computer Society and member of the discipline evaluation group of Shanghai Academic Degrees Committee. He was admitted to the computer department of Nanjing University in 1985 and graduated with a Bachelor of Science Degree in 1989. He received a master's degree from East China University of technology in 1992 and a doctor's degree from Shanghai Jiaotong University in 1995. He began to teach in East China University of technology in 1995, and engaged in visiting research in the school of computer science of Florida International University from 2001 to 2004. He is mainly engaged in cutting-edge research in the fields of software engineering, trusted computing, big data and cloud computing. He has presided over four scientific research projects supported by the National Natural Science Foundation of China, as well as a number of funded projects such as the Ministry

14680394, 2022, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.12924 by Ned Univ of Engineering & Tech, Wiley Online Library on [08/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

of education's key teachers' funding plan for colleges and universities, the special scientific research fund for doctoral disciplines, Shanghai dawn plan, Shanghai Pujiang talent plan and so on, He has published more than 200 papers in academic journals and international conferences at home and abroad, compiled 2 teaching materials, and won one Shanghai Science and technology progress award.

**Chen Liqiong** is a professor at Shanghai Institute of applied technology. He received a doctorate from East China University of technology in March 2009, majoring in computer application technology, and his research interests are software engineering, distributed real-time embedded system, formal method, trusted computing, etc. From 2006 to 2010, he participated in the National Natural Science Foundation Project "component development method of embedded system based on concept refinement", mainly responsible for the model design and verification technology of embedded system; During this period, he also participated in the "Research on the development of intelligent agent technology in the field of information service" of Shanghai Science and technology development fund, mainly responsible for the design, analysis and implementation technology of the model. In 2009, he won the excellent youth fund of Shanghai Institute of Applied Technology and continued to engage in the design and research of distributed real-time embedded systems. This paper mainly studies the construction and verification of traffic control model. In recent years, he has published more than 30 academic papers at home and abroad, including 1 SCI search and more than 20 EI search. In 2012, he won the support of Shanghai industry university research project, went to enterprises for industry university research practice, studied the key technologies of the Internet of things and applied them to the modern logistics industry.

**How to cite this article:** Liu, Z., Fan, G., Yu, H., & Chen, L. (2022). Modelling and analysing the reliability for microservice-based cloud application based on predicate Petri net. *Expert Systems*, *39*(6), e12924. https://doi.org/10.1111/exsy.12924

14680394, 2022, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/exsy.12924 by Ned Univ of Engineering & Tech, Wiley Online Library on [08/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License